

Segunda versão do projeto da disciplina

Comparação entre os algoritmos de ordenação elementar

1. Introdução

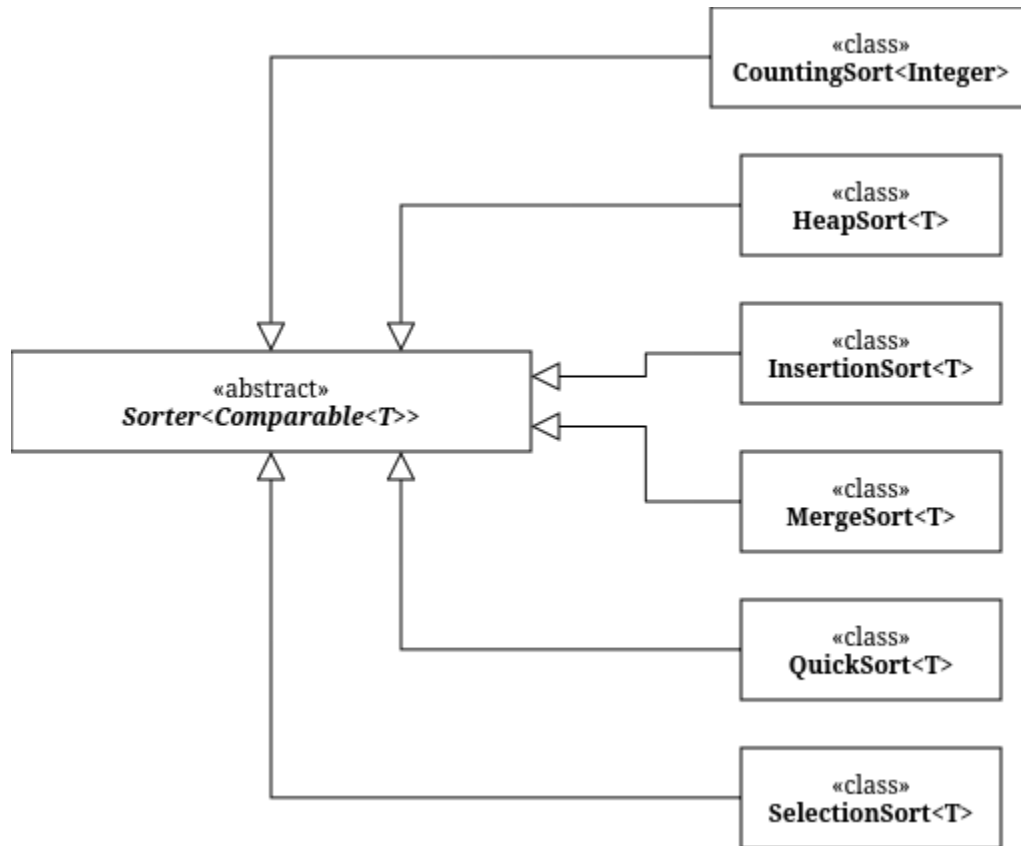
Este relatório corresponde ao relato dos resultados obtidos no projeto da disciplina de LEDA que tem por objetivo fazer uma análise comparativa entre os algoritmos de ordenação elementares.

Para esse fim, foi implementado um projeto de exploração de dados no qual através da aplicação prática dos algoritmos de ordenação sobre um conjunto de dados real foi possível verificar a performance desses algoritmos.

Na segunda versão do projeto foram implementados tipos abstratos de dados, substituindo os diversos usos de *Arrays* de dados usados para armazenamento.

2. Descrição geral sobre o método utilizado

A ferramenta foi implementada em Java versão 1.8, usando a plataforma *maven* para automação de *build*. Foi usada uma superclasse abstrata como base para a implementação dos algoritmos de ordenação.



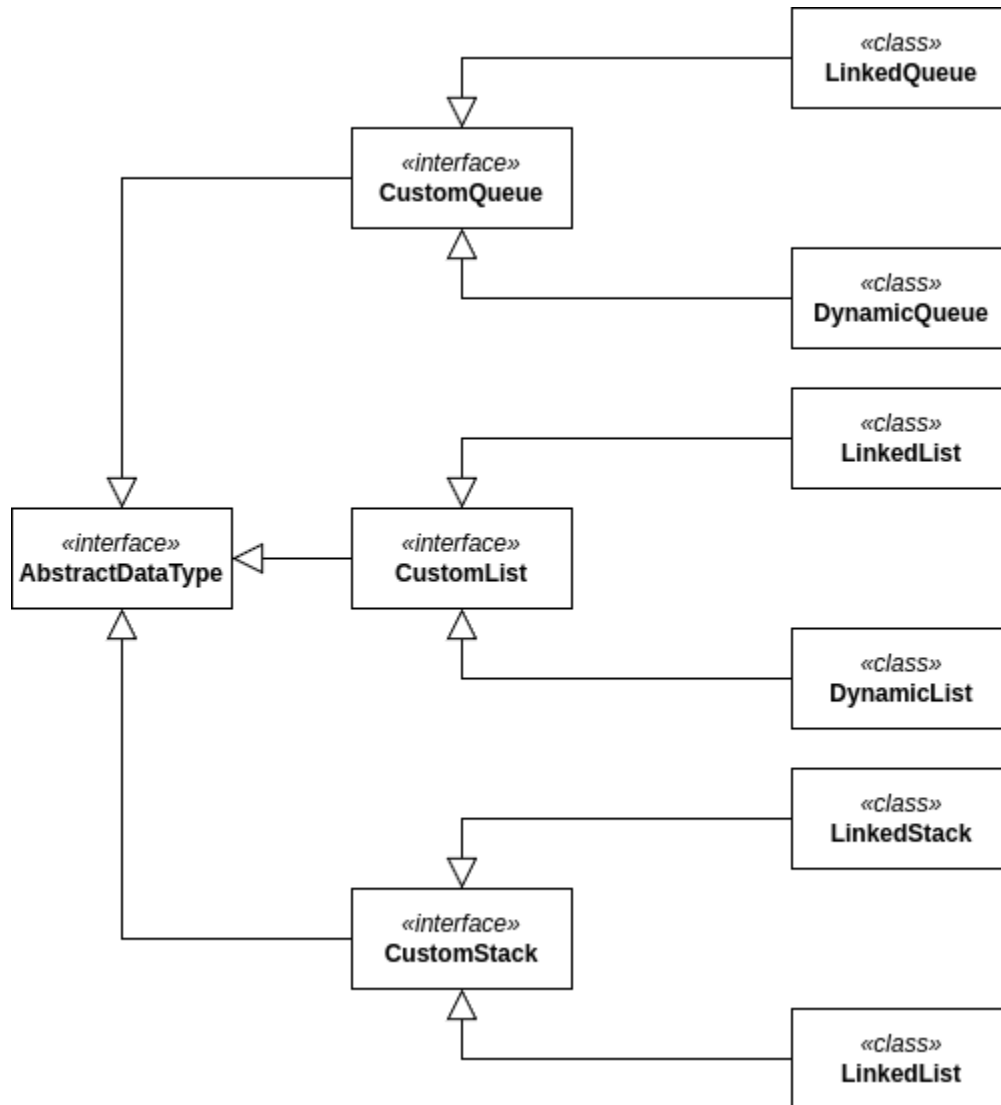
Implementação dos algoritmos de ordenação

As classes derivadas da classe abstrata *Sorter* processam *arrays* de tipos genéricos comparáveis, e retornam um array de inteiros que indicam a posição de cada elemento.

A ordenação dos dados ocorre através do processamento de colunas, as quais são extraídas do arquivo em disco e ordenadas usando os métodos de ordenação. O *array* de inteiros resultante é usado para reorganizar as linhas do arquivo original, o qual é salvo novamente para o disco.

Na segunda versão foram implementados os tipos de dados abstratos lista, fila e pilha. Cada e todo uso de *array* fora da implementação dos dados abstratos foi substituído por uma instância de um tipo abstrato de dados.

Todos os tipos abstratos de dados herdam da interface *AbstractDataType*. Os dados são divididos em duas categorias: os baseados em *arrays* dinâmicos e os baseados em *nodos simplesmente encadeados*.



Implementação dos tipos abstratos de dados.

Foram usadas listas encadeadas na conversão de tabelas em disco para outras estruturas.

Listas dinâmicas foram usadas internamente nos processos de ordenação, no qual era necessário maior rapidez de troca e obtenção, ao custo do maior uso de memória.

Pilhas foram mais usadas quando fosse necessário inverter a ordem de alguma coleção de dados, como por exemplo na obtenção de ordem decrescente.

O caso principal de uso das filas é na reordenação dos arquivos, servindo como saída dos algoritmos de ordenação.

Descrição geral do ambiente de testes

Foram utilizados dois sistemas: um para desenvolvimento/testes e outro somente para testes.

O primeiro sistema é do tipo *notebook*, com o processador AMD A12-9720P de quatro núcleos e arquitetura *x86-64*, 8 GB (*gigabytes*) de memória e 120 GB de armazenamento em disco. O sistema operacional utilizado foi o *linux*, versão 6.11.0, distribuição CachyOS com Java OpenJDK. Foi utilizado para desenvolvimento e testes.

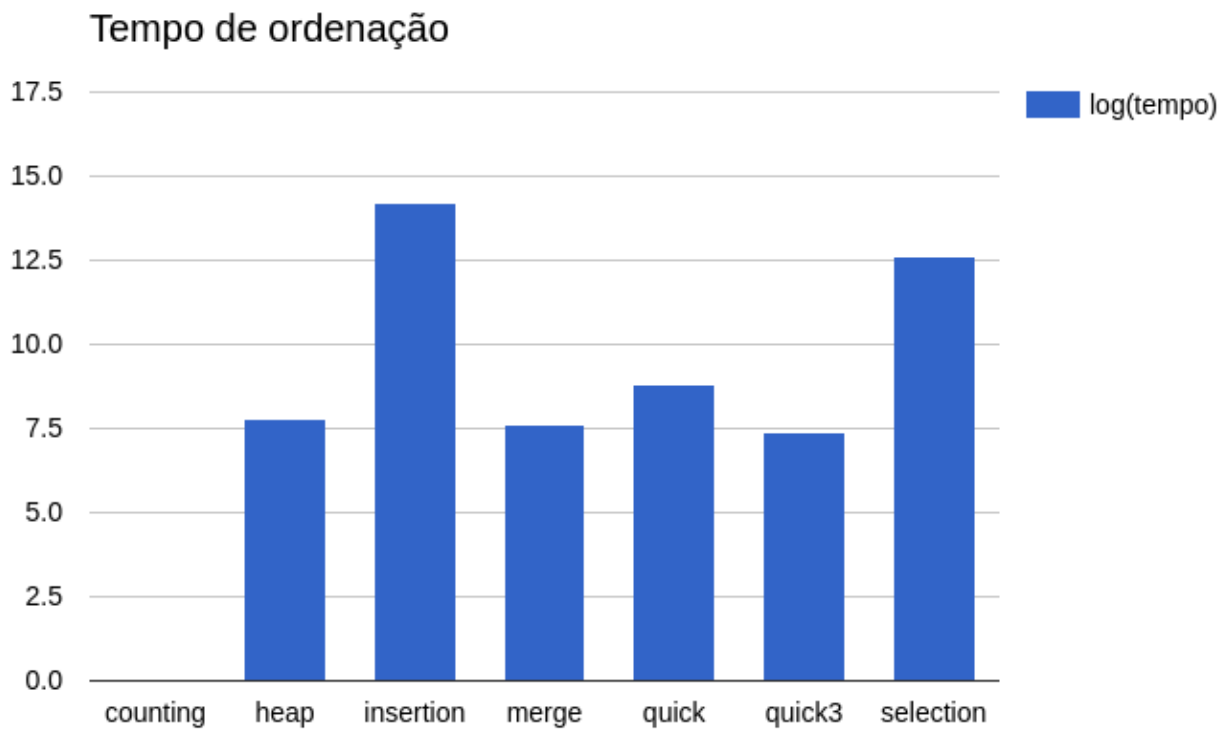
O segundo sistema utilizado foi um computador do tipo desktop com o processador AMD Athlon 3000G, de dois núcleos e arquitetura *x86-64*, 4 GB (*gigabytes*) de memória e 500 GB de armazenamento em disco, com sistema operacional Windows 10. Foi utilizado a JRE proprietária da Oracle. O sistema foi utilizado para testes.

3. Resultados e Análise

Resultados da primeira versão

Ordenação pela data do acidente (coluna *date*) em ordem decrescente. Tempo de execução medido em milissegundos.

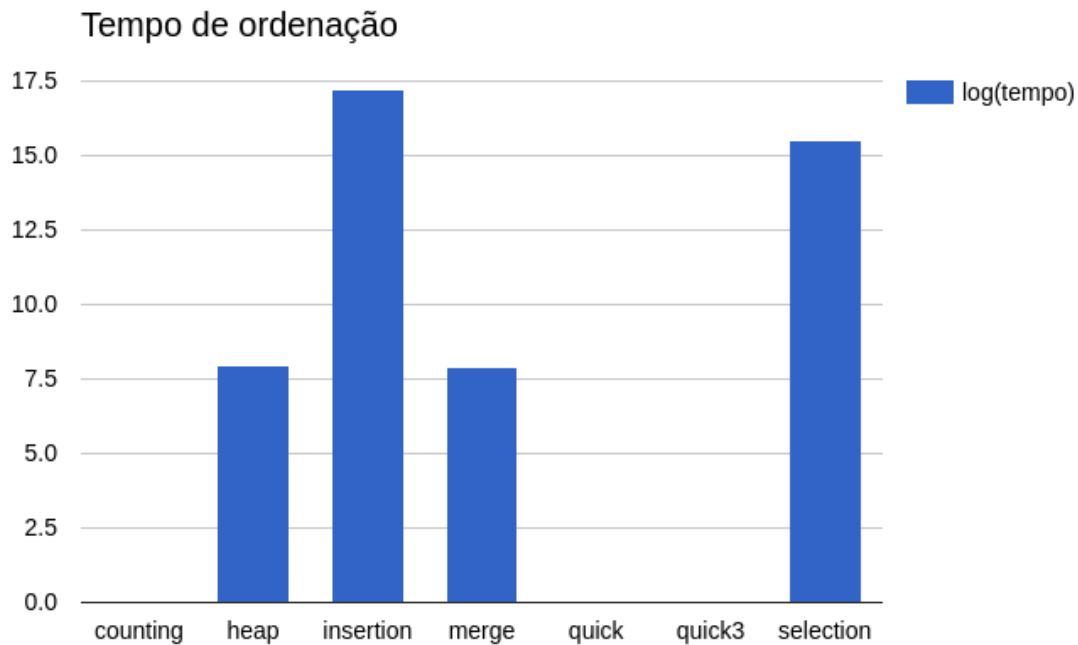
Counting	Heap	Insertion	Merge	Quick	Quick-Mediana de 3	Selection
Não aplicável	2396 ms	1508713 ms	2047 ms	6568 ms	1614 ms	2999972 ms



O gráfico reforça a disparidade em eficiência dos métodos recursivos sobre os métodos tradicionais. Dentre os métodos recursivos, o *quick-sort* foi o mais demorado, justificado pela variação de complexidade entre caso médio $O(\log n)$ e pior caso $O(n^2)$.

Ordenação pela coluna *time* em ordem crescente. Tempo de execução medido em milissegundos.

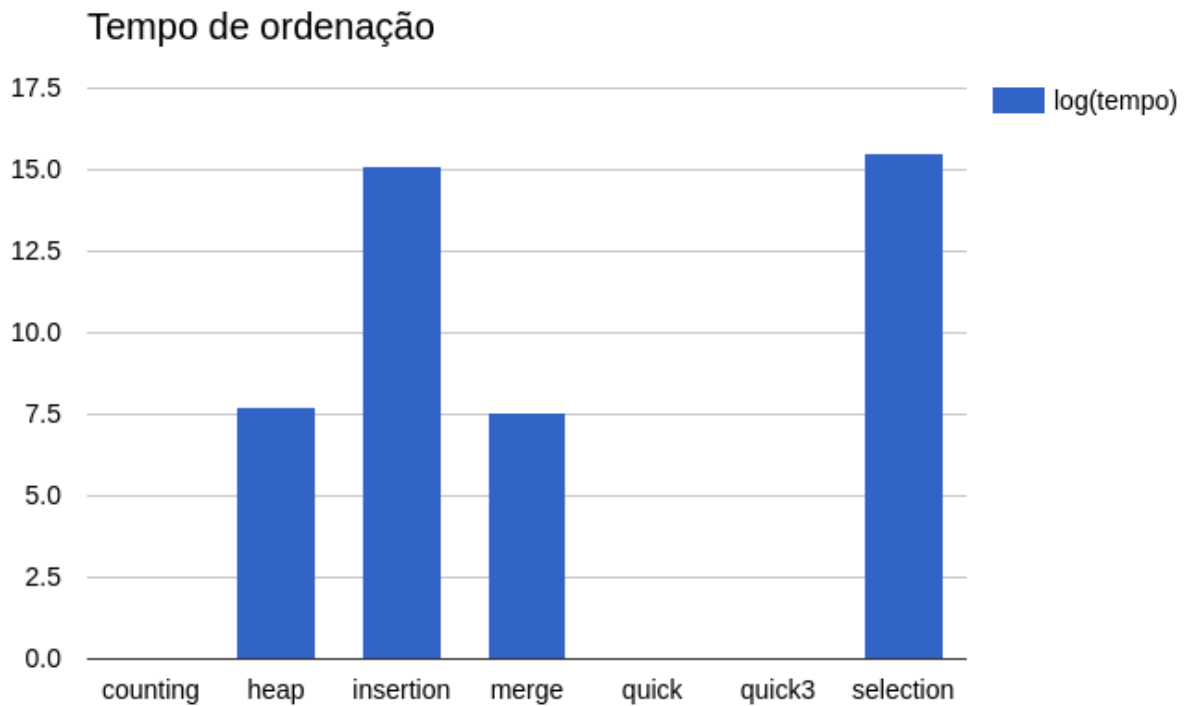
Counting	Heap	Insertion	Merge	Quick	Quick-Mediana de 3	Selection
Não Aplicável	2805 ms	30422157 ms	2687 ms	Estouro da Stack	Estouro da Stack	5409667 ms



Mais uma vez os algoritmos *heap-sort* e *merge-sort* são rápidos. A variação de uso de espaço no *quick-sort* aparente pelo *StackOverflowError* durante a execução, provavelmente devido à partição desbalanceada causada pelo algoritmo *partition()*.

Ordenação pelo tipo de comunicação (coluna *communication_kind*) em ordem alfabética crescente. Tempo de execução medido em milissegundos.

Counting	Heap	Insertion	Merge	Quick	Quick-Mediana de 3	Selection
Não Aplicável	2330 ms	3784142 ms	1895 ms	Estouro da Stack	Estouro da Stack	5430091 ms



Os resultados demonstram a eficiência dos algoritmos recursivos sobre os iterativos, sobretudo do algoritmo *merge-sort*. O algoritmo *heap-sort* foi o segundo melhor em termo de rapidez de execução, seguido pelo *quick-sort*, o pior dentre os algoritmos recursivos devido à variação de complexidade de uso de espaço e tempo que o algoritmo apresenta. O *insertion-sort* foi o melhor algoritmo iterativo, e o *selection-sort* o pior devido ao excesso de comparações necessárias para se obter o menor elemento em cada iteração, o que causa o caso médio de execução de se aproximar do pior caso mais frequentemente.

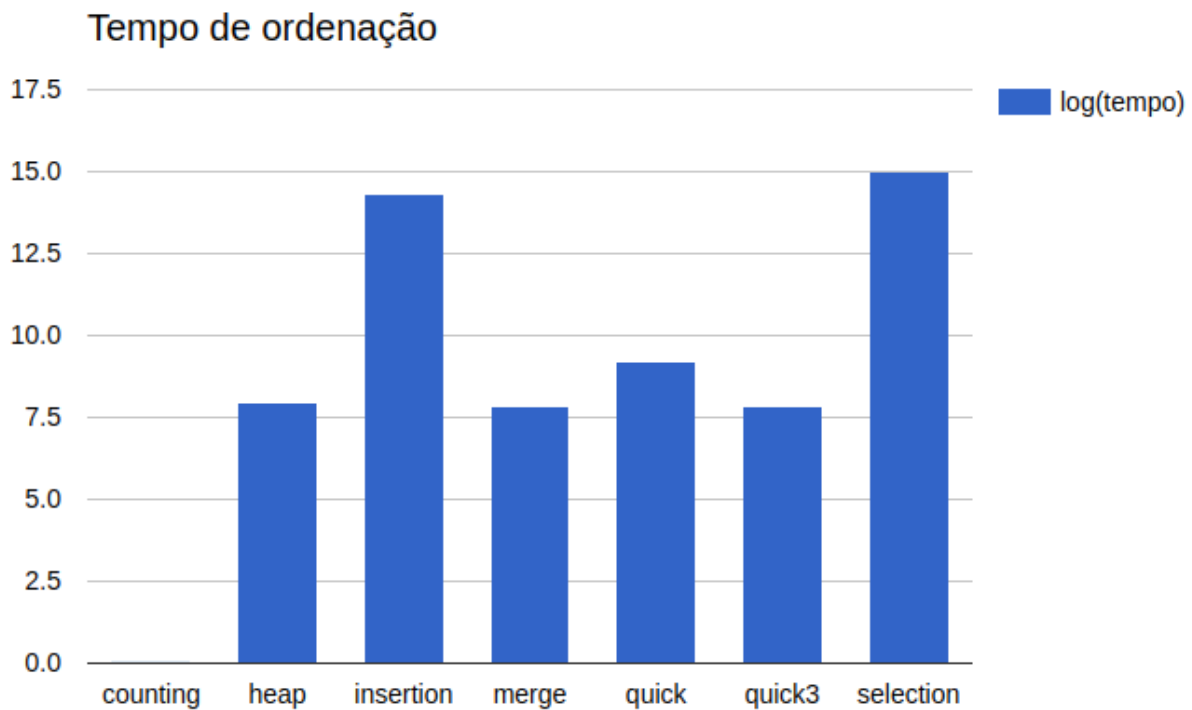
Resultados da segunda versão

Com uma implementação alternativa do algoritmo recursivo do *quick-sort* e *quick-sort* com mediana de 3, foi possível completar a ordenação sem estouro da *stack*. Porém a implementação da estratégia de *buckets* não foi possível por não atender a todo e qualquer tipo genérico comparável.

De modo geral, é esperado que haja um aumento de tempo e de uso de memória devido ao uso de estruturas de dados complexas ao invés de *arrays* simples.

1 - Ordenação pela data do acidente (coluna *date*) em ordem decrescente. Tempo de execução medido em milissegundos.

Counting	Heap	Insertion	Merge	Quick	Quick-Mediana de 3	Selection
Não aplicável	2831 ms	1656686 ms	2510 ms	9803 ms	2600 ms	3226035 ms



De modo consistente com o da primeira versão, os algoritmos iterativos foram os que levaram mais tempo, com *insertion-sort* à frente do *selection-sort*. Dentre os algoritmos recursivos, *merge-sort* foi o mais rápido, mas não muito mais que *quick-sort* com mediana de 3.

2 - Ordenação pela coluna *time* em ordem crescente. Tempo de execução medido em milissegundos.

Counting	Heap	Insertion	Merge	Quick	Quick-Mediana de 3	Selection
2640 ms	3543 ms	5186993 ms	1975 ms	175990 ms	159065 ms	5226152 ms



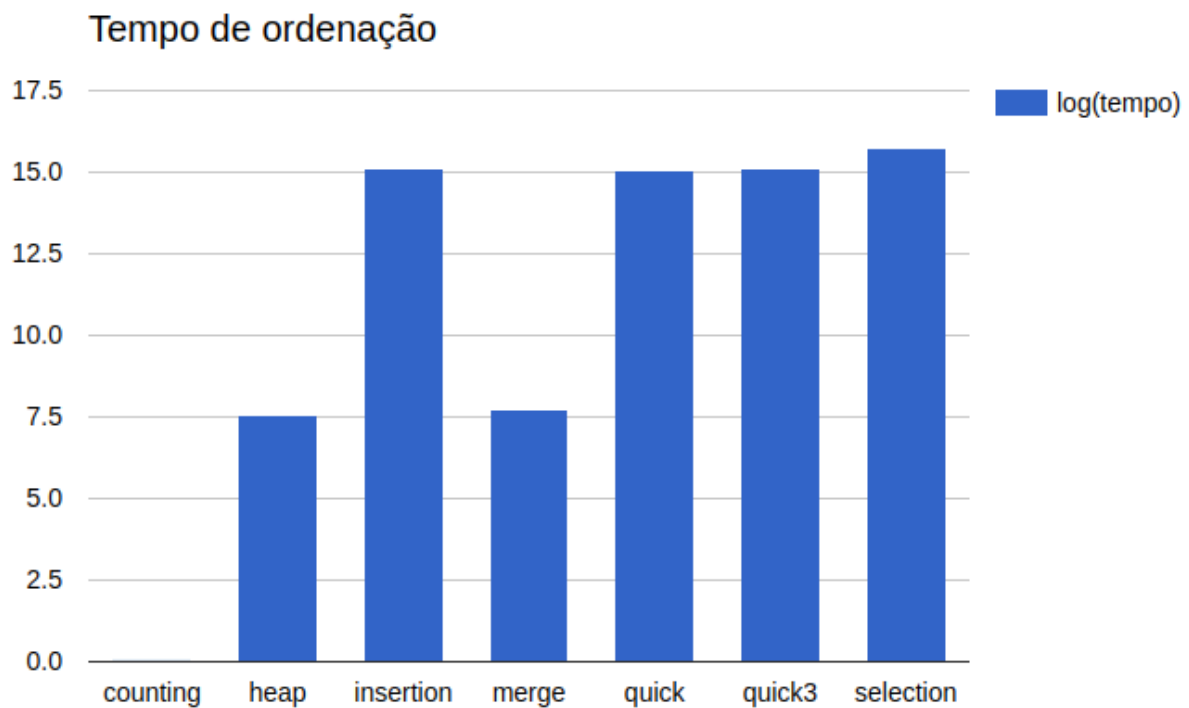
O algoritmo *merge-sort* foi o mais rápido, seguido pelo *heap-sort*. O *quick-sort* e *quick-sort* com mediana de 3 foram os mais lentos dentre os algoritmos recursivos, devido à tendência de pior caso $O(n^2)$ ao ordenar elementos repetidos ou parcialmente ordenados.

O *counting-sort* foi o mais rápido dentre os não-recursivos, devido a não executar comparações.

Insertion-sort e *selection-sort* tiveram praticamente o mesmo tempo de execução, e os piores tempos, esperado dos algoritmos iterativos.

3 - Ordenação pelo tipo de comunicação (coluna *communication_kind*) em ordem alfabética crescente. Tempo de execução medido em milissegundos.

Counting	Heap	Insertion	Merge	Quick	Quick-Mediana de 3	Selection
Não Aplicável	1904 ms	3599545 ms	2253 ms	3428779 ms	3604755 ms	6953641 ms



Novamente o *quick-sort* e sua variante mostram o pior caso ao ordenar muitos elementos parecidos, sendo quase piores até que os algoritmos iterativos.

Heap-sort obteve o menor tempo, marginalmente melhor que o *merge-sort*.